FinGPT

Project Description:

Welcome to FinGPT - an AI-augmented personal finance tracker that helps users record income and expenses, monitor savings, set category budgets, and understand spending patterns from one structured workspace.

Built with the MERN stack (MongoDB, Express.js, React, and Node.js), FinGPT combines monthly transaction ledgers, charts, PDF reporting, and a Gemini-powered assistant. The AI layer categorizes transactions, produces grounded monthly insights, and answers budgeting questions using the authenticated user's stored financial data. Amounts are presented in Indian Rupees.

FinGPT turns day-to-day financial records into clear, practical decisions without spreadsheet clutter.

Scenario-Based Case Study:

A salaried professional wants to control household spending but currently records transactions across notes and spreadsheets. At the end of each month, it is difficult to see how much was earned, where money was spent, whether category limits were exceeded, and what practical action should be taken next.

The Solution:

FinGPT provides a secure monthly finance workspace where the user records income and expenses, reviews calculated totals and charts, sets category budgets, exports a report, and receives AI-generated guidance grounded in the same stored records.

How FinGPT Helps Users

- **Secure Registration and Login -** Create an account, sign in with hashed credentials, and use JWT-backed application sessions.
- **Monthly Transaction Tracking -** Record income and expense rows by month and maintain them with add, edit, and delete actions.
- **Automatic Financial Totals -** Recalculate total income, total expense, and current savings after each change.
- **AI Categorization and Insights -** Use Gemini to classify transactions and generate short, data-backed observations.
- **Budget Planning -** Set monthly limits by category and compare budgeted, spent, remaining, and over-budget amounts.

- **Charts and Reports -** Review monthly trends and category distribution, then download a PDF summary.
- **Grounded Budgeting Assistant -** Ask finance questions and receive concise answers based on real stored monthly data.

System Requirements:

The following software and hardware requirements support local development, testing, and deployment of the FinGPT web application.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or a modern Linux distribution.
- **Node.js:** A current LTS release compatible with Vite 7 and the server dependencies.
- **npm:** Used to install and run frontend and backend packages.
- **React 18 + Vite:** Frontend application and development build tool.
- **Express.js 4:** REST API and middleware framework.
- **MongoDB:** Local MongoDB or MongoDB Atlas for users, monthly expenses, and budgets.
- **Gemini API key:** Required for transaction categorization, insights, and assistant chat.
- **Browser:** Latest Google Chrome, Microsoft Edge, or Firefox.
- **Postman or Insomnia:** Recommended for API testing.
- **Visual Studio Code and Git:** Recommended editor and version-control tools.

2. Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or better recommended for smooth local development.
- **Memory:** 8 GB minimum; 16 GB recommended when running the client, server, database, and browser together.
- **Storage:** At least 1 GB free for source code, dependencies, builds, and local database data.
- **Display:** 1920 x 1080 recommended for dashboard and responsive-layout validation.

Project Architecture:

FinGPT uses a modular client-server architecture:

React + Vite client <==> Express + Node.js REST API <==> MongoDB, with Gemini AI and PDFKit as integrated services.

Technical Architecture:

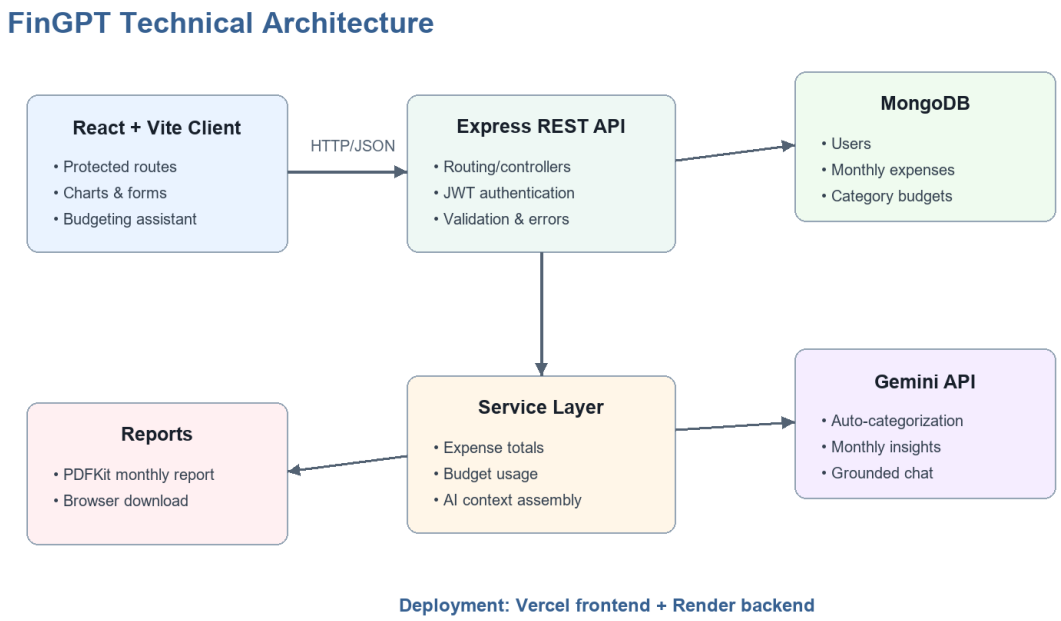

Figure 1. FinGPT technical architecture

The frontend communicates with route groups mounted by the Express application. Controllers validate request data and delegate reusable work to services. Mongoose models persist users, monthly expense documents, and category budgets. The AI service uses Gemini for structured categorization and insights plus a conversational budgeting assistant.

1. User Interface (UI)

The React 18 frontend uses Vite, React Router, Bootstrap, styled-components, Recharts, and a Three.js finance scene. Protected routes expose Home, Track, Add Transaction, Budgets, and Assistant pages after authentication.

2. Web Server

The Node.js and Express backend parses JSON and URL-encoded requests, enables CORS, connects to MongoDB, and mounts authentication, user, expense, report, AI, and budget routes.

3. API Gateway / Router

Route modules map HTTP endpoints to controllers for login and registration, monthly transaction CRUD, PDF generation, budget operations, and AI requests.

4. Authentication Service

Registration hashes passwords with bcrypt. Login validates the password and returns a signed JWT plus the user record. The frontend stores the session through cookies for protected navigation.

5. Database (MongoDB)

Mongoose schemas store users, monthly expense ledgers with embedded transaction rows, and monthly category budgets. Expense calculations are recomputed after writes.

6. AI and Reporting Services

Gemini categorizes transaction rows, creates structured insight cards, and answers chat questions from a compact financial snapshot. PDFKit streams a monthly report to the browser.

ER Diagram:

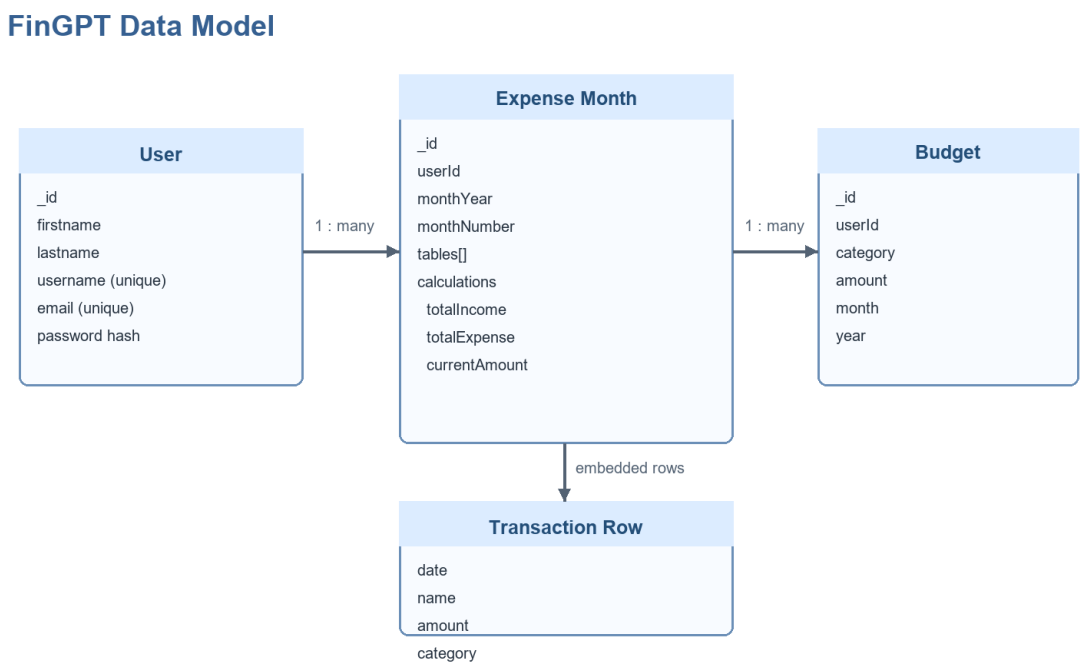

Figure 2. FinGPT document relationships

1. User and Expense Relationship

Type: one-to-many. Each monthly Expense document stores the user's identifier, month/year, transaction tables, and calculated totals.

2. User and Budget Relationship

Type: one-to-many. Each Budget document belongs to a user and is uniquely scoped by category, month, and year.

3. Expense and Transaction Row Relationship

Type: embedded one-to-many. Each monthly document contains INCOME and EXPENSE tables whose rows store date, name, amount, and optional AI category.

Features

- **Role-aware session flow:** Visitors can register or log in; authenticated users access finance routes and can log out.
- **Monthly ledger management:** Add new month data, filter by month/year, edit or delete individual rows, and delete an entire month.
- **Calculated finance summary:** Total income, total expense, and current savings remain synchronized with transaction rows.
- **Interactive analytics:** Recharts renders a monthly trend bar chart and category spending pie chart.
- **Spending guardrail:** The tracker warns when total expenses exceed half of monthly income.
- **AI transaction categorization:** Gemini assigns controlled income or expense categories and writes them back to the month.
- **AI insight cards:** The application returns a short summary and 3-5 severity-tagged insights grounded in real numbers.
- **Budget monitoring:** Monthly category budgets report spent, remaining, percentage used, and over-budget state.
- **PDF report export:** A downloadable report lists monthly transactions and totals.
- **Deployment readiness:** Render and Vercel configuration files support split backend/frontend deployment.

Roles and Responsibilities

1. Visitor

- **Account access:** Register a new account or log in with an existing email and password.
- **Public navigation:** Use authentication pages; protected pages redirect when no JWT cookie is present.

2. Registered User

- **Transaction management:** Create, review, edit, and delete monthly income and expense entries.
- **Financial review:** Inspect totals, trends, category distribution, and spending alerts.
- **Budget planning:** Set and remove monthly category limits and monitor use.
- **AI assistance:** Categorize rows, generate insight cards, and ask grounded budgeting questions.
- **Reporting:** Download a PDF copy of the selected monthly ledger.

User Flow

Registered User Journey

Figure 3. Registered user journey

MVC Pattern

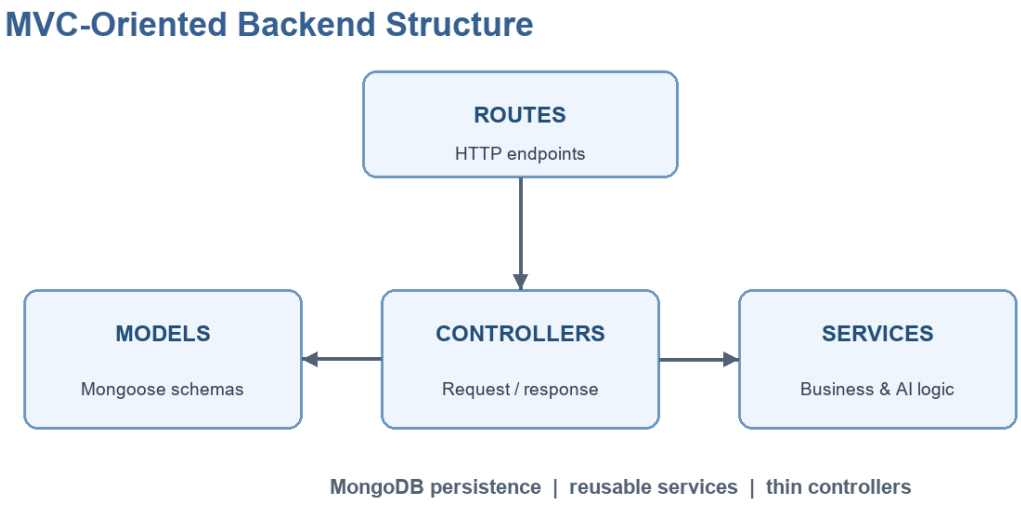

Figure 4. Backend route-controller-service-model flow

FinGPT follows an MVC-oriented backend with an additional service layer. Models define data, controllers handle request/response responsibilities, routes expose endpoints, and services contain financial calculations, budget aggregation, and Gemini integration.

Project Setup and Configuration

Creating the Project Folder

- **1. Clone the repository.** Use Git to obtain the project locally.
- **2. Open the root folder.** The source contains separate client/client and server/server applications.
- **3. Install dependencies.** Run npm install inside both nested application folders.
- **4. Create environment files.** Copy each .env.example to .env and fill only local/deployment-specific values.

Backend Development:

Backend Structure:

server/server

```
server.js
src/app.js
src/db/connect.js
src/models/user.model.js
src/models/expense.model.js
src/models/budget.model.js
src/controllers/
src/services/expense.service.js
src/services/budget.service.js
src/services/ai.service.js
src/routes/
src/utils/pdfReport.js
.env.example
package.json
```

Figure 5. Backend project structure

- **server.js:** Loads environment variables, imports the Express app, and listens on PORT (default 5100).
- **src/app.js:** Configures middleware, database startup, and route mounting.
- **controllers/:** Coordinates authentication, expense, budget, AI, user, and report responses.
- **services/:** Contains reusable calculations, budget aggregation, and Gemini workflows.
- **models/:** Defines User, Expense, and Budget Mongoose schemas.
- **routes/:** Declares REST endpoints and maps them to controllers.
- **utils/pdfReport.js:** Streams the selected month as a PDFKit report.

API Route Groups

src/app.js - mounted API groups

```
app.use('/', authRoutes);
app.use('/', userRoutes);
app.use('/', expenseRoutes);
app.use('/', reportRoutes);
app.use('/ai', aiRoutes);
app.use('/budgets', budgetRoutes);
```

Figure 6. Route groups mounted by the Express application

Database Development:

1. Configure MongoDB

Set MONGO_URI in server/server/.env. The supplied example defaults to mongodb://127.0.0.1:27017/exp for local development, while deployment can use MongoDB Atlas.

src/db/connect.js

```
const mongoose = require('mongoose');
require('dotenv').config();
const db = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/exp';
mongoose.connect(db, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => console.log('Connection successful'))
  .catch((error) => console.log(`No connection: ${error}`));
```

Figure 7. MongoDB connection module

2. Configure Environment Variables

- **MONGO_URI:** MongoDB connection string.
- **PORT:** Backend port; defaults to 5100.
- **GEMINI_API_KEY:** Google Gemini credential used only by the server.
- **GEMINI_MODEL:** Model identifier; the project example uses gemini-2.5-flash.
- **JWT_SECRET:** Long random key used to sign login tokens.
- **VITE_API_URL:** Frontend base URL for the deployed or local backend.

3. Create Schemas

User schema: first name, last name, username, email, and bcrypt password hash.

Expense schema: user/month identity, transaction tables, embedded rows, and derived totals.

Budget schema: user, category, amount, month, and year with a unique compound index.

Front-End Development:

Structure:

client/client/src

```
App.js
theme.css
utils/api.js
components/Home/
components/Track/
components/AddIncomeAndExpense/
components/BudgetManager/
components/BudgetAssistant/
components/AiInsights/
components/Charts/
components/Login/
components/Registration/
components/ProtectedRoute/
components/Header/
```

Figure 8. Frontend component structure

- **App.js:** Defines BrowserRouter routes and lazy-loads the global Three.js finance backdrop.
- **Home/:** Landing dashboard with finance overview, CTA links, and a 3D finance visual.
- **Track/:** Monthly ledger, charts, row CRUD, AI tools, PDF export, and calculated summaries.
- **AddIncomeAndExpense/:** Form for monthly income or expense entry with category selection.
- **BudgetManager/:** Monthly category budget form, progress calculations, and deletion.
- **BudgetAssistant/:** Gemini chat interface grounded in the signed-in user's data.
- **AiInsights/ and Charts/:** Reusable AI insight cards and Recharts visualizations.
- **utils/api.js:** Normalizes VITE_API_URL and produces consistent endpoint URLs.

Project Execution:

Step 1: Start the Backend Server

```
cd server/server
npm install
copy .env.example .env
npm start
```

The backend listens at http://localhost:5100 unless PORT is changed.

Step 2: Start the Frontend

```
cd client/client
npm install
copy .env.example .env
npm start
```

Vite serves the frontend at http://localhost:3000 by default in the project instructions. Ensure VITE_API_URL points to the backend.

Step 3: Deployment

- **Backend on Render:** Use server/server as the root, npm install as the build command, npm start as the start command, and configure the four server environment variables.
- **Frontend on Vercel:** Use client/client as the root, npm run build, dist as the output, and set VITE_API_URL to the Render service URL.

Output Screenshots:

Home / Dashboard Page:

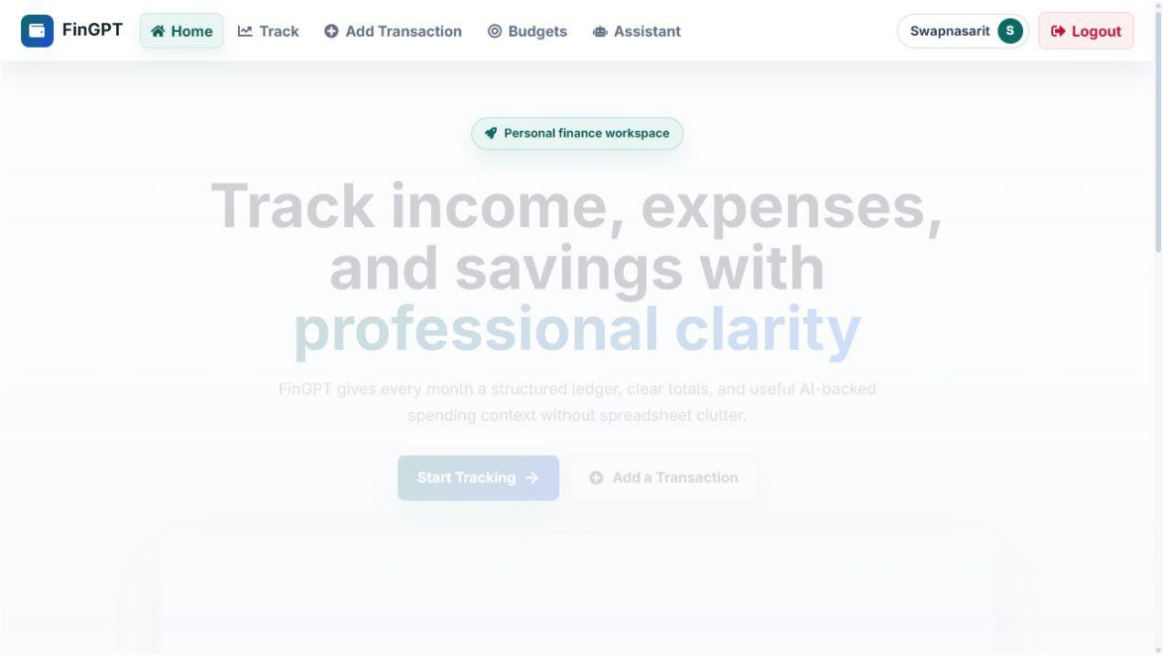

Figure 9. Home / Dashboard Page

Monthly Tracker Page:

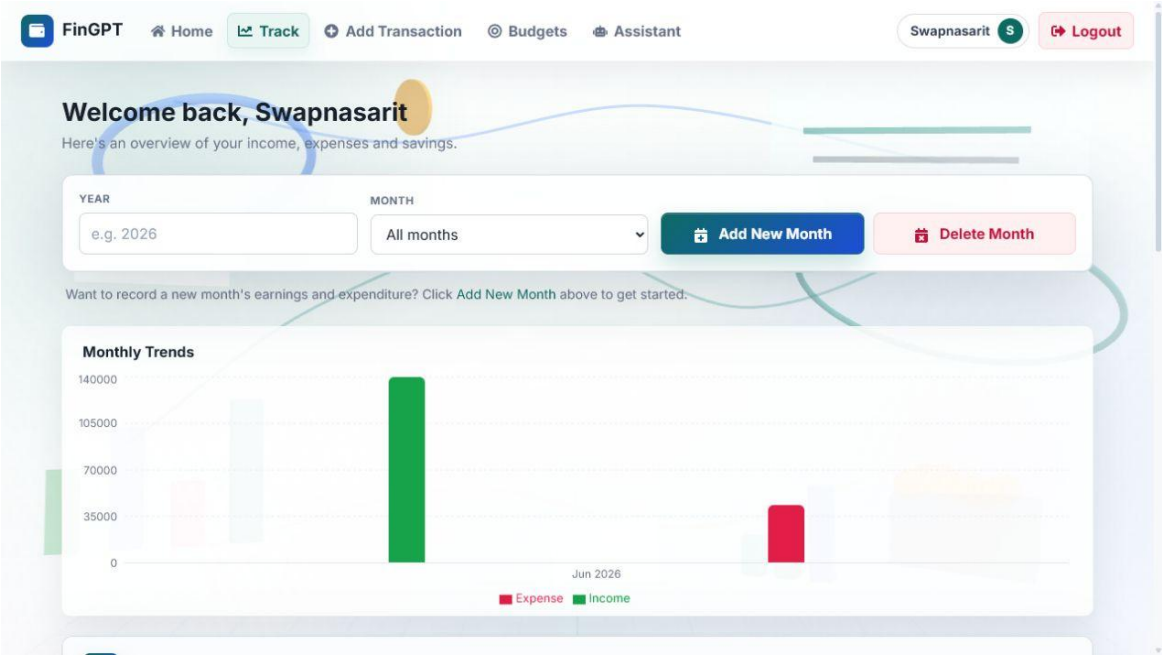

Figure 10. Monthly Tracker Page

Add Transaction Page:

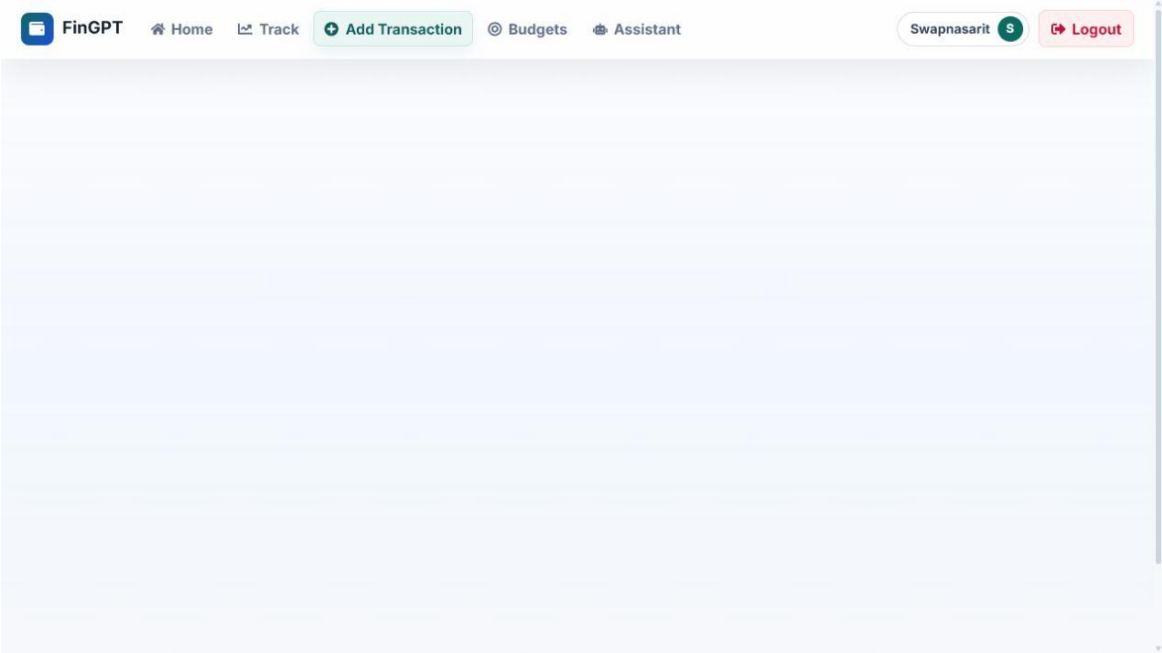

Figure 11. Add Transaction Page

Budget Manager Page:

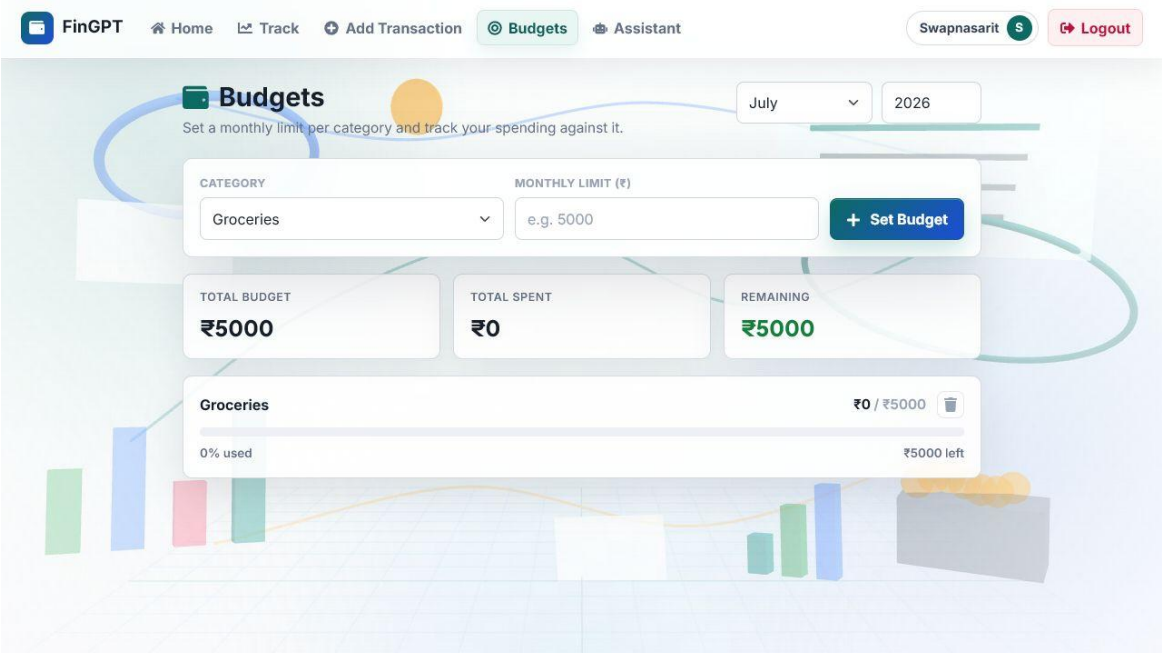

Figure 12. Budget Manager Page

AI Budgeting Assistant Page:

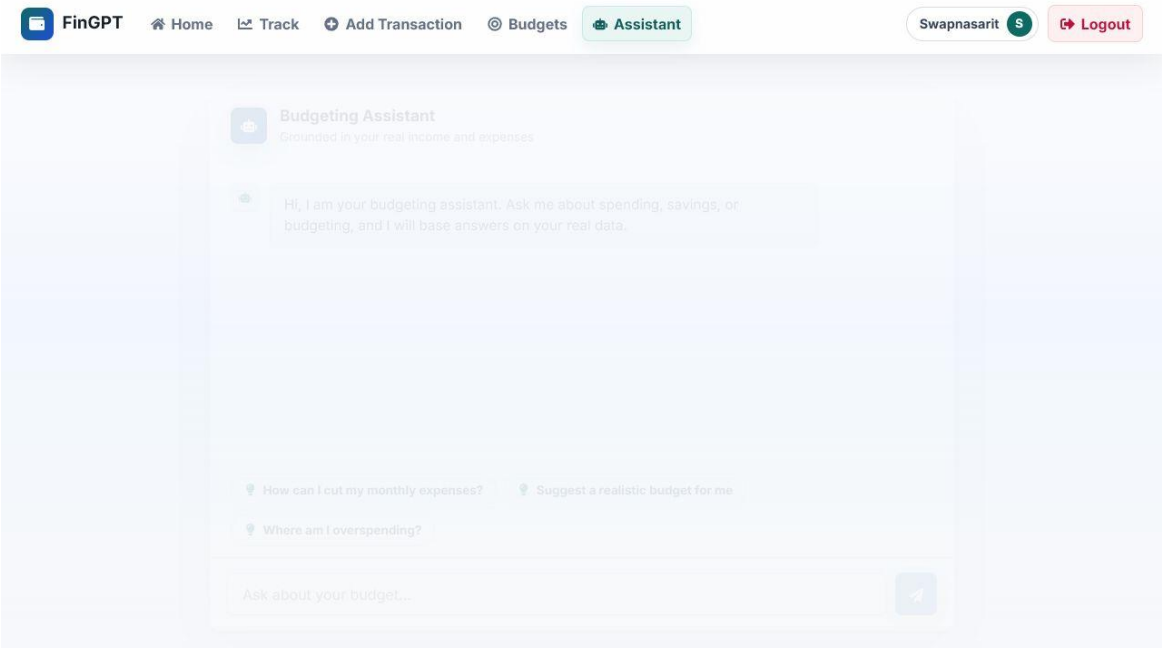

Figure 13. AI Budgeting Assistant Page

Code Repository

GitHub: https://github.com/sarit-smartbridge/FinGpt-Finance-Tracker